Spectral Degeneracies of a Topmost-to-Random Shuffle on Balanced Binary Words

A note based on discussions with Professor Steven N. Evans

2026-11-03

State space

The state space is the set of words of length $2n$ with n letters R and n letters G, so that the number of states is $\binom{2n}{n}$. Equivalently, a state is a subset of the $2n$ positions occupied by R, and the chain is a walk on the n-subsets of $\{1, \dots, 2n\}$. In the notation of the thesis, this is $\mathcal{S}_n = \{\sigma \in \{R, G\}^{2n} : \#R(\sigma) = \#G(\sigma) = n\}$ with $|\mathcal{S}_n| = \binom{2n}{n}$.

Transitions

We identify the topmost card of a deck with the leftmost letter of the word, so that the current word is $x_1 x_2 \cdots x_{2n}$ with x_1 as the topmost (leftmost) letter. At each step, this topmost letter is removed and inserted uniformly at random into one of the $2n$ slots defined by the remaining $2n-1$ letters — a "topmost-to-random" (equivalently, leftmost-to-random) insertion, as opposed to the usual top-to-random shuffle on a deck of distinct cards. Equivalently, remove x_1 to leave $y = x_2 \cdots x_{2n}$, and choose uniformly $j \in \{0, 1, \dots, 2n-1\}$; the new word is y with x_1 inserted after the j-th symbol of y (with $j = 0$ meaning at the front, i.e. returned to the top). Each slot is chosen with probability $1/(2n)$, so every row of the transition matrix has exactly $2n$ entries (some possibly equal, if two slots produce the same word), each equal to $1/(2n)$. In the notation of the thesis, $P_n = (P_n(\sigma, \tau))_{\sigma, \tau \in \mathcal{S}_n}$ and $P_n(\sigma, \tau) = \frac{1}{2n} \cdot \#\{\text{insertion positions that turn } \sigma \text{ into } \tau\}$.

The transition matrix

n = 1

The states are RG and GR, so $\binom{2}{1} = 2$. Take the word RG: the topmost letter is R, and removing it leaves G, which defines the two slots G (before G and after G). Inserting R into slot 1 returns RG, and inserting into slot 2 gives GR, each with probability 1/2. Take the word GR: the topmost letter is G, and removing it leaves R, with slots R (before R and after R). Inserting G into slot 1 returns GR, and inserting into slot 2 gives RG, each with probability 1/2. In R/G form:

- RG $\to$ RG (prob 1/2), RG $\to$ GR (prob 1/2);
- GR $\to$ GR (prob 1/2), GR $\to$ RG (prob 1/2).

The complement map $R \leftrightarrow G$ sends RG $\leftrightarrow$ GR, so the space splits into a symmetric block (vectors with $v_{\mathrm{RG}} = v_{\mathrm{GR}}$) and an antisymmetric block (vectors with $v_{\mathrm{RG}} = -v_{\mathrm{GR}}$), each of dimension 1. The symmetric block is [1] and the antisymmetric block is [0], and assembling them gives the transition matrix in bracket form:

$$P = \begin{pmatrix} \frac{1}{2} & \frac{1}{2} \\ \frac{1}{2} & \frac{1}{2} \end{pmatrix}.$$

n = 2

The states are RRGG, RGRG, RGGR, GRRG, GRGR, GGRR, in lexicographic order with R < G, so $\binom{4}{2} = 6$. Take each state in turn and remove its topmost letter, leaving three letters that define four slots.

- **RRGG**: remove R, leaving RGG with slots *R_G_G*. Slot 1 gives RRGG, slot 2 gives RRGG, slot 3 gives RGRG, slot 4 gives RGGR. So RRGG → RRGG (2/4), RGRG (1/4), RGGR (1/4).
- **RGRG**: remove R, leaving GRG with slots *G_R_G*. Slot 1 gives RGRG, slot 2 gives GRRG, slot 3 gives GRRG, slot 4 gives GRGR. So RGRG → RGRG (1/4), GRRG (2/4), GRGR (1/4).
- **RGGR**: remove R, leaving GGR with slots *G_G_R*. Slot 1 gives RGGR, slot 2 gives GRGR, slot 3 gives GGRR, slot 4 gives GGRR. So RGGR → RGGR (1/4), GRGR (1/4), GGRR (2/4).
- **GRRG**: remove G, leaving RRG with slots *R_R_G*. Slot 1 gives GRRG, slot 2 gives RGRG, slot 3 gives RRGG, slot 4 gives RRGG. So GRRG → GRRG (1/4), RGRG (1/4), RRGG (2/4).
- **GRGR**: remove G, leaving RGR with slots *R_G_R*. Slot 1 gives GRGR, slot 2 gives RGGR, slot 3 gives RGGR, slot 4 gives RGRG. So GRGR → GRGR (1/4), RGGR (2/4), RGRG (1/4).
- **GGRR**: remove G, leaving GRR with slots *G_R_R*. Slot 1 gives GGRR, slot 2 gives GGRR, slot 3 gives GRGR, slot 4 gives GRRG. So GGRR → GGRR (2/4), GRGR (1/4), GRRG (1/4).

The complement map sends RRGG ↔ GGRR, RGRG ↔ GRGR, RGGR ↔ GRRG. Choosing one representative from each pair (RRGG, RGRG, RGGR) gives a 3×3 symmetric block and a 3×3 antisymmetric block, and the full 6×6 matrix splits as $P = P_s \oplus P_a$ after the complement pairing. Assembling the two blocks gives the transition matrix in bracket form:

$$P = \frac{1}{4} \begin{pmatrix} 2 & 1 & 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 2 & 1 & 0 \\ 0 & 0 & 1 & 0 & 1 & 2 \\ 2 & 1 & 0 & 1 & 0 & 0 \\ 0 & 1 & 2 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 & 1 & 2 \end{pmatrix}.$$

n = 3

The states are RRRGGG, RRGRGG, RRGGRG, RRGGGR, RGRRGG, RGRGRG, RGRGGR, RGGRRG, RGGRGR, RGGGRR, GRRRGG, GRRGRG, GRRGGR, GRGRRG, GRGRGR, GRGGRR, GGRRRG, GGRRGR, GGRGRR, GGGRRR, in lexicographic order with R < G, so $\binom{6}{3} = 20$. Removing the topmost letter of each state leaves five letters that define six slots. The transitions, in R/G form, are:

- RRRGGG → RRRGGG (2/6), RRGRGG (1/6), RRGGRG (1/6), RRGGGR (2/6);
- RRGRGG → RRGRGG (2/6), RGRRGG (1/6), RGRGRG (1/6), RGRGGR (2/6);
- RRGGRG → RRGGRG (2/6), RGRGRG (1/6), RGGRRG (1/6), RGGRGR (2/6);
- RRGGGR → RRGGGR (2/6), RGRGGR (1/6), RGGRGR (1/6), RGGGRR (2/6);
- RGRRGG → RGRRGG (1/6), GRRRGG (2/6), GRRGRG (1/6), GRRGGR (2/6);
- RGRGRG → RGRGRG (1/6), GRRGRG (1/6), GRGRRG (2/6), GRGRGR (2/6);
- RGRGGR → RGRGGR (1/6), GRRGGR (1/6), GRGRGR (1/6), GRGGRR (3/6);
- RGGRRG → RGGRRG (1/6), GRGRRG (1/6), GGRRRG (2/6), GGRRGR (2/6);
- RGGRGR → RGGRGR (1/6), GRGRGR (1/6), GGRRGR (1/6), GGRGRR (3/6);
- RGGGRR → RGGGRR (1/6), GRGGRR (1/6), GGRGRR (1/6), GGGRRR (3/6);
- GRRRGG → RRRGGG (3/6), RRGRGG (1/6), RGRRGG (1/6), GRRRGG (1/6);
- GRRGRG → RRGRGG (3/6), RRGGRG (1/6), RGRGRG (1/6), GRRGRG (1/6);
- GRRGGR → RRGGRG (2/6), RRGGGR (2/6), RGRGGR (1/6), GRRGGR (1/6);
- GRGRRG → RGRRGG (3/6), RGRGRG (1/6), RGGRRG (1/6), GRGRRG (1/6);
- GRGRGR → RGRGRG (2/6), RGRGGR (2/6), RGGRGR (1/6), GRGRGR (1/6);
- GRGGRR → RGGRRG (2/6), RGGRGR (1/6), RGGGRR (2/6), GRGGRR (1/6);
- GGRRRG → GRRRGG (2/6), GRRGRG (1/6), GRGRRG (1/6), GGRRRG (2/6);
- GGRRGR → GRRGRG (2/6), GRRGGR (1/6), GRGRGR (1/6), GGRRGR (2/6);
- GGRGRR → GRGRRG (2/6), GRGRGR (1/6), GRGGRR (1/6), GGRGRR (2/6);
- GGGRRR → GGRRRG (2/6), GGRRGR (1/6), GGRGRR (1/6), GGGRRR (2/6).

The complement map sends state s to $21 - s$: RRRGGG ↔ GGGRRR, RRGRGG ↔ GGRGRR, RRGGRG ↔ GGRRGR, RRGGGR ↔ GGRRRG, RGRRGG ↔ GRGGRR, RGRGRG ↔ GRGRGR, RGRGGR ↔ GRGRRG, RGGRRG ↔ GRRGGR, RGGRGR ↔ GRRGRG, RGGGRR ↔ GRRRGG. Choosing one representative from each pair gives the ten representatives RRRGGG, RRGRGG, RRGGRG, RRGGGR, RGRRGG, RGRGRG, RGRGGR, RGGRRG, RGGRGR, RGGGRR, so the symmetric block is 10×10 and the antisymmetric block is 10×10, and the full 20×20 matrix splits as $P = P_s \oplus P_a$ after the complement pairing. The symmetric block P_s acts on vectors with

$v_s = v_{21-s}$, and the antisymmetric block P_a acts on vectors with $v_s = -v_{21-s}$; a transition from s to t contributes $+\frac{1}{6}$ to P_s and $\mathrm{sgn}(t)\frac{1}{6}$ to P_a, where $\mathrm{sgn}(t) = +1$ if $t \le 21 - t$ and -1 otherwise. Assembling the two blocks gives the transition matrix in bracket form:

$$P = \frac{1}{6}\begin{bmatrix}
2 & 1 & 1 & 2 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & 2 & 0 & 0 & 1 & 1 & 2 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 2 & 0 & 0 & 1 & 0 & 1 & 2 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 2 & 0 & 0 & 1 & 0 & 1 & 2 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 2 & 1 & 2 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 2 & 2 & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 1 & 3 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 2 & 2 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 1 & 3 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 1 & 3 \\
3 & 1 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & 3 & 1 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 2 & 2 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 3 & 1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 2 & 2 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 2 & 1 & 2 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 2 & 1 & 0 & 1 & 0 & 0 & 2 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 2 & 1 & 0 & 1 & 0 & 0 & 2 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 2 & 1 & 1 & 0 & 0 & 2 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 2 & 1 & 1 & 2
\end{bmatrix}$$

Because $P = P_s \oplus P_a$, the spectrum of P is the union of the spectra of the two blocks, and the block decomposition is what makes the dense diagonalization feasible. The directed graph $\mathcal{G}_4$ for $n = 2$, shown below, makes the transition structure visible: vertices are the six states, an edge $\sigma \to \tau$ has weight $P_4(\sigma, \tau)$, self-loops occur with probability at least $1/4$, and every column sums to 1, confirming double stochasticity.

```
import networkx as nx
import matplotlib.pyplot as plt
import numpy as np
from matplotlib.patches import FancyArrowPatch, Circle

states = ['RRGG', 'RGRG', 'RGGR', 'GRRG', 'GRGR', 'GGRR']
pos = nx.circular_layout(states)

# Construct the transition matrix directly from the insertion rule.
state_to_idx = {state: i for i, state in enumerate(states)}
P = np.zeros((len(states), len(states)))

for i, state in enumerate(states):
    top = state[0]
    rest = state[1:]
    for slot in range(len(state)):
        new_state = rest[:slot] + top + rest[slot:]
```

```
        P[i, state_to_idx[new_state]] += 1 / len(state)

# Generate graph edges directly from the transition matrix.
edges = [
    (states[i], states[j], P[i, j])
    for i in range(len(states))
    for j in range(len(states))
    if P[i, j] > 0
]

assert np.allclose(P.sum(axis=1), 1), "Rows do not sum to 1."
assert np.allclose(P.sum(axis=0), 1), "Columns do not sum to 1."

def format_prob(w):
    if np.isclose(w, 1/2):
        return r"$\frac{1}{2}$"
    if np.isclose(w, 1/4):
        return r"$\frac{1}{4}$"
    return f"{w:.2f}"


fig, ax = plt.subplots(figsize=(9, 8))
pos = nx.circular_layout(states, scale=1.15)

EDGE_WIDTH = 2.0
EDGE_COLOR = "#4a148c"
ARROW_SIZE = 20

edge_set = {(u, v) for u, v, w in edges}
curves = []

# Draw nodes.
for node in states:
    x, y = pos[node]

    ax.add_patch(Circle(
        (x, y),
        radius=0.15,
        facecolor="#e0f7fa",
        edgecolor="#006064",
        linewidth=2,
        zorder=4
    ))

    ax.text(
        x, y, node,
        fontsize=9,
        fontweight="bold",
        ha="center",
        va="center",
        zorder=5
    )

# Draw every directed transition independently.
for u, v, w in edges:

    x1, y1 = pos[u]
```

```
    x2, y2 = pos[v]

    if u == v:
        angle = np.arctan2(y1, x1)

        loop = FancyArrowPatch(
            (x1 + 0.10*np.cos(angle + 0.35),
             y1 + 0.10*np.sin(angle + 0.35)),
            (x1 + 0.10*np.cos(angle - 0.35),
             y1 + 0.10*np.sin(angle - 0.35)),
            connectionstyle="arc3,rad=1.8",
            arrowstyle="-|>",
            mutation_scale=ARROW_SIZE,
            color=EDGE_COLOR,
            linewidth=EDGE_WIDTH,
            zorder=2
        )
        ax.add_patch(loop)

        distance = 0.25 if u in ("RRGG", "GRRG") else 0.32

        curves.append({
            "u": u, "v": v, "w": w,
            "loop": True,
            "label_pos": (
                x1 + distance*np.cos(angle),
                y1 + distance*np.sin(angle)
            )
        })
        continue

    reverse = (v, u) in edge_set

    # Reciprocal transitions receive separate arcs.
    rad = 0.30 if reverse else 0.12

    # Draw the curved arc without an arrowhead.
    arc = FancyArrowPatch(
        (x1, y1),
        (x2, y2),
        connectionstyle=f"arc3,rad={rad}",
        arrowstyle="-",
        color=EDGE_COLOR,
        linewidth=EDGE_WIDTH,
        alpha=0.95,
        shrinkA=22,
        shrinkB=22,
        zorder=2
    )
    ax.add_patch(arc)

    # Quadratic Bezier representation of the arc.
    dx = x2 - x1
    dy = y2 - y1

    cx = (x1 + x2)/2 + rad*dy
    cy = (y1 + y2)/2 - rad*dx
```

```
ts = np.linspace(0, 1, 101)

curve_x = (
    (1-ts)**2*x1
    + 2*(1-ts)*ts*cx
    + ts**2*x2
)

curve_y = (
    (1-ts)**2*y1
    + 2*(1-ts)*ts*cy
    + ts**2*y2
)

# Place the arrowhead at exactly 3/4 of the Bezier arc.
t_arrow = 0.85

arrow_x = (
    (1-t_arrow)**2*x1
    + 2*(1-t_arrow)*t_arrow*cx
    + t_arrow**2*x2
)

arrow_y = (
    (1-t_arrow)**2*y1
    + 2*(1-t_arrow)*t_arrow*cy
    + t_arrow**2*y2
)

tangent_x = (
    2*(1-t_arrow)*(cx-x1)
    + 2*t_arrow*(x2-cx)
)

tangent_y = (
    2*(1-t_arrow)*(cy-y1)
    + 2*t_arrow*(y2-cy)
)

tangent_norm = np.hypot(tangent_x, tangent_y)
tangent_x /= tangent_norm
tangent_y /= tangent_norm

arrow_length = 0.055

ax.annotate(
    "",
    xy=(arrow_x, arrow_y),
    xytext=(
        arrow_x - arrow_length*tangent_x,
        arrow_y - arrow_length*tangent_y
    ),
    arrowprops=dict(
        arrowstyle="-|>",
        color=EDGE_COLOR,
        lw=EDGE_WIDTH,
```

```
            mutation_scale=ARROW_SIZE
        ),
        zorder=3
    )

    curves.append({
        "u": u, "v": v, "w": w,
        "loop": False,
        "x": curve_x,
        "y": curve_y
    })


# ------------------------------------------------------
# Collision-aware label placement.
# ------------------------------------------------------

fig.canvas.draw()
renderer = fig.canvas.get_renderer()

node_boxes = []

for node in states:
    x, y = pos[node]
    center = ax.transData.transform((x, y))
    radius = abs(
        ax.transData.transform((x + 0.15, y))[0] - center[0]
    )

    node_boxes.append((
        center[0] - radius - 4,
        center[1] - radius - 4,
        center[0] + radius + 4,
        center[1] + radius + 4
    ))


def boxes_overlap(a, b, padding=3):
    return not (
        a.x1 + padding < b.x0 or
        a.x0 - padding > b.x1 or
        a.y1 + padding < b.y0 or
        a.y0 - padding > b.y1
    )


placed_boxes = []


# Place self-loop labels first.
for item in curves:
    if not item["loop"]:
        continue

    x, y = item["label_pos"]

    label = ax.text(
```

```
        x, y,
        format_prob(item["w"]),
        fontsize=14,
        fontweight="bold",
        color="#24103f",
        ha="center",
        va="center",
        bbox=dict(
            facecolor="white",
            edgecolor="#8054aa",
            linewidth=1.1,
            boxstyle="round,pad=0.4"
        ),
        zorder=9
    )

    fig.canvas.draw()
    placed_boxes.append(label.get_window_extent(renderer))


# Place non-loop labels using a collision score.
for item in curves:

    if item["loop"]:
        continue

    curve_display = ax.transData.transform(
        np.column_stack((item["x"], item["y"]))
    )

    # Alternate label positions along the 12 directed arcs.
    # Cycle through quarter/three-quarter positions and
    # alternate between opposite sides of each arc.

    placement_pattern = [
        (2/5,  1),
        (1/4, -1),
        (1/3,  1),
        (3/4, -1),
        (1/7,  1)
    ]

    arc_index = sum(
        1 for previous in curves[:curves.index(item)]
        if not previous["loop"]
    )

    t, side = placement_pattern[arc_index % 5]

    index = int(t * 100)
    point = curve_display[index]

    tangent = (
        curve_display[min(index + 1, 100)]
        - curve_display[max(index - 1, 0)]
    )
```

```
    tangent_norm = np.linalg.norm(tangent)

    if tangent_norm < 1e-8:
        continue

    tangent = tangent / tangent_norm
    normal = np.array([-tangent[1], tangent[0]])

    candidates = []

    for offset in (18, 24, 30, 36, 42):

        candidate = point + side * offset * normal

        candidates.append((
            candidate,
            t,
            side,
            offset
        ))

    best = None
    best_score = float("inf")

    for candidate, t, side, offset in candidates:

        data_position = ax.transData.inverted().transform(
            candidate
        )

        trial = ax.text(
            data_position[0],
            data_position[1],
            format_prob(item["w"]),
            fontsize=14,
            fontweight="bold",
            color="#24103f",
            ha="center",
            va="center",
            bbox=dict(
                facecolor="white",
                edgecolor="#8054aa",
                linewidth=1.1,
                boxstyle="round,pad=0.4"
            ),
            zorder=9
        )

        fig.canvas.draw()
        bbox = trial.get_window_extent(renderer)

        score = abs(t - 0.5) * 100

        # Avoid every node.
        for nb in node_boxes:
            if (
                bbox.x0 < nb[2] and bbox.x1 > nb[0]
```

```
                and bbox.y0 < nb[3] and bbox.y1 > nb[1]
            ):
                score += 100000

        # Avoid previously placed labels.
        for pb in placed_boxes:
            if boxes_overlap(bbox, pb, padding=5):
                score += 100000

        # Avoid crossing other arcs.
        center = np.array([
            (bbox.x0 + bbox.x1)/2,
            (bbox.y0 + bbox.y1)/2
        ])

        half_diagonal = 0.5*np.hypot(
            bbox.width, bbox.height
        )

        for other in curves:
            if other is item or other["loop"]:
                continue

            other_display = ax.transData.transform(
                np.column_stack((other["x"], other["y"]))
            )

            distances = np.linalg.norm(
                other_display - center, axis=1
            )

            clearance = np.min(distances) - half_diagonal

            if clearance < 9:
                score += (9 - clearance)**2 * 20

        # Prefer labels close to their own arc.
        score += abs(offset - 18)*0.5

        if score < best_score:
            best_score = score
            best = (data_position, bbox)

        trial.remove()

    if best is None:
        continue

    data_position, _ = best

    label = ax.text(
        data_position[0],
        data_position[1],
        format_prob(item["w"]),
        fontsize=14,
        fontweight="bold",
        color="#24103f",
```

```
        ha="center",
        va="center",
        bbox=dict(
            facecolor="white",
            edgecolor="#8054aa",
            linewidth=1.1,
            boxstyle="round,pad=0.4"
        ),
        zorder=9
    )

    fig.canvas.draw()
    placed_boxes.append(label.get_window_extent(renderer))


ax.set_title(
    "Directed graph $\\mathcal{G}_4$",
    fontsize=16,
    pad=24
)

ax.set_xlim(-1.65, 1.65)
ax.set_ylim(-1.65, 1.65)
ax.set_aspect("equal")
ax.axis("off")

plt.tight_layout()
plt.show()
```

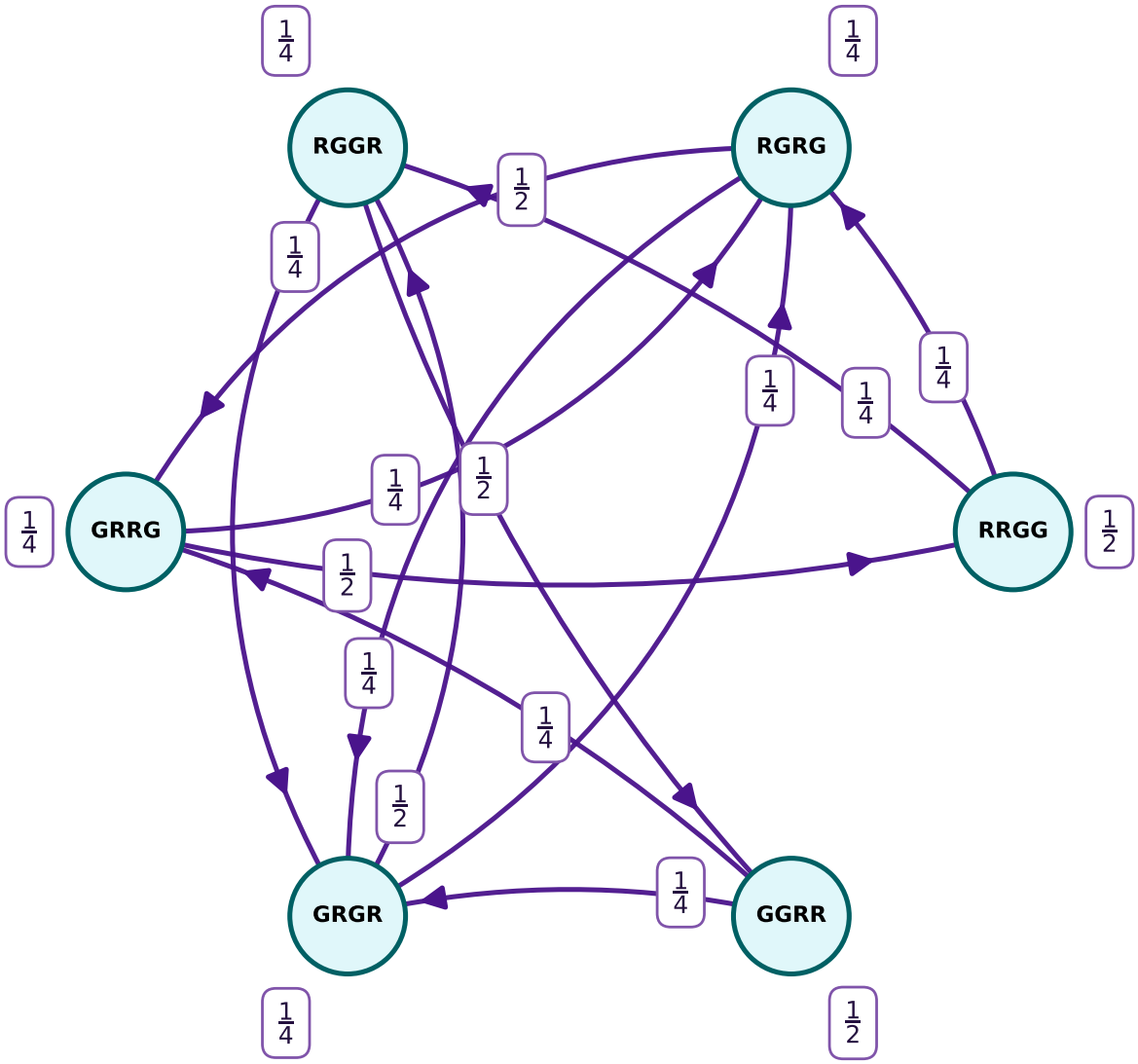


Figure 1: Directed graph $\mathcal{G}_4$ of the colored top-to-random chain (edge width proportional to weight). Every column sums to 1, so P_4 is doubly stochastic.

True eigenvalues

Because $P = P_s \oplus P_a$, the spectrum of P is the union of the spectra of the two blocks. I diagonalized both blocks densely and took the union. For $n = 3$ the true multiplicities are

$$(m_0, m_2, m_3, m_4, m_5, m_6) = (1, 2, 3, 4, 4, 6),$$

summing to $\binom{6}{3} = 20$; for $n = 9$ they are

$$(m_0, \dots, m_{18}) = (1, 0, 1, 2, 4, 8, 16, 32, 64, 128, 256, 510, 1002, 1914, 3498, 6006, 9438, 12870, 1),$$

summing to $\binom{18}{9} = 48{,}620$. In every case the true eigenvalues are exactly

$$1 - \frac{k}{2n}, \qquad k = 0, 2, 3, \dots, 2n,$$

with $k = 1$ absent for all $n \geq 1$. The figure below plots the spectrum for $n = 4, 6, 8$, obtained by diagonalizing the full transition matrix P_n (not just the blocks) for each n. The horizontal bands are the values $1 - k/(2n)$ with $k = 0, 2, 3, \dots, 2n$; the band at $1 - 1/(2n)$ is absent, and the multiplicities grow with n as the band thickens.

```
import numpy as np
import matplotlib.pyplot as plt
from itertools import combinations

def build_transition_matrix(n):
    states = []
    for indices in combinations(range(n), n//2):
        s = ['0'] * n
        for i in indices:
            s[i] = '1'
        states.append(''.join(s))
    state_to_idx = {s: i for i, s in enumerate(states)}
    N = len(states)
    P = np.zeros((N, N))
    for i, state in enumerate(states):
        top = state[0]
        for pos in range(n):
            new_state = list(state[1:])
            new_state.insert(pos, top)
            new_state = ''.join(new_state)
            P[i, state_to_idx[new_state]] += 1.0 / n
    return P

fig, ax = plt.subplots(figsize=(7, 5))
for n in [4, 6, 8]:
    P = build_transition_matrix(n)
    evs = np.sort(np.linalg.eigvals(P).real)[::-1]
    ax.scatter(range(len(evs)), evs, label=f'n={n}', s=50, alpha=0.7)
ax.set_xlabel('Index k')
ax.set_ylabel('Eigenvalue')
ax.set_title('Eigenvalues of P_n')
ax.legend()
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

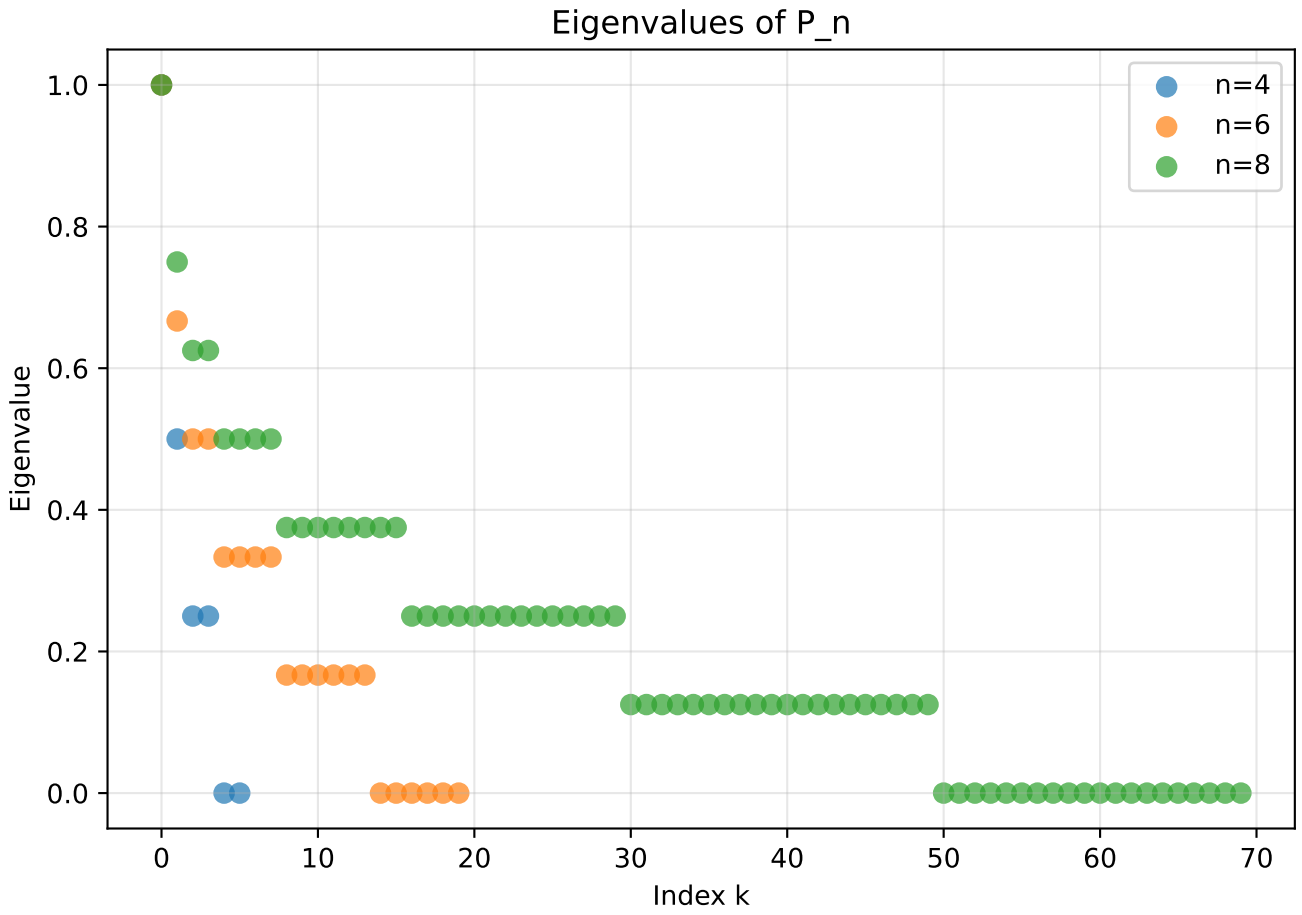


Figure 2: Eigenvalues of P_n for $n \in \{4, 6, 8\}$. The bands are the surviving values $1 - k/(2n)$; the $k = 1$ band is absent.

Computational limit

I ran the computation for $n = 1, \ldots, 9$. The $n = 9$ case (48,620 states) already took considerable time and memory on my laptop, and $n = 10$ ($\binom{20}{10} = 184{,}756$ states) was not attempted because it exceeded the available memory. The state count $\binom{2n}{n}$ is well approximated by $4^n/\sqrt{\pi n}$, as the figure below shows, which is why the dense $n \leq 9$ computation is the practical limit: at $n = 10$ the symmetry blocks are $92{,}378 \times 92{,}378$, beyond the memory of a laptop. This is the practical limit of the dense block diagonalization on the hardware at hand; the trace method below scales further but I have not pushed it past $n = 9$.

```
import math
import matplotlib.pyplot as plt

ns = list(range(4, 20, 2))
exact = [math.comb(2*n, n) for n in ns]
stirling = [(4**n) / math.sqrt(math.pi * n) for n in ns]

fig, ax = plt.subplots(figsize=(7, 5))
ax.plot(ns, exact, 'o-', label='Exact $\\binom{2n}{n}$')
ax.plot(ns, stirling, 'x--', label='Stirling $4^n/\\sqrt{\\pi n}$')
ax.set_xlabel('n')
ax.set_ylabel('Value')
ax.set_title('Stirling approximation vs. exact')
ax.legend()
ax.grid(True, alpha=0.3)
```

```
ax.set_yscale('log')
plt.tight_layout()
plt.show()
```

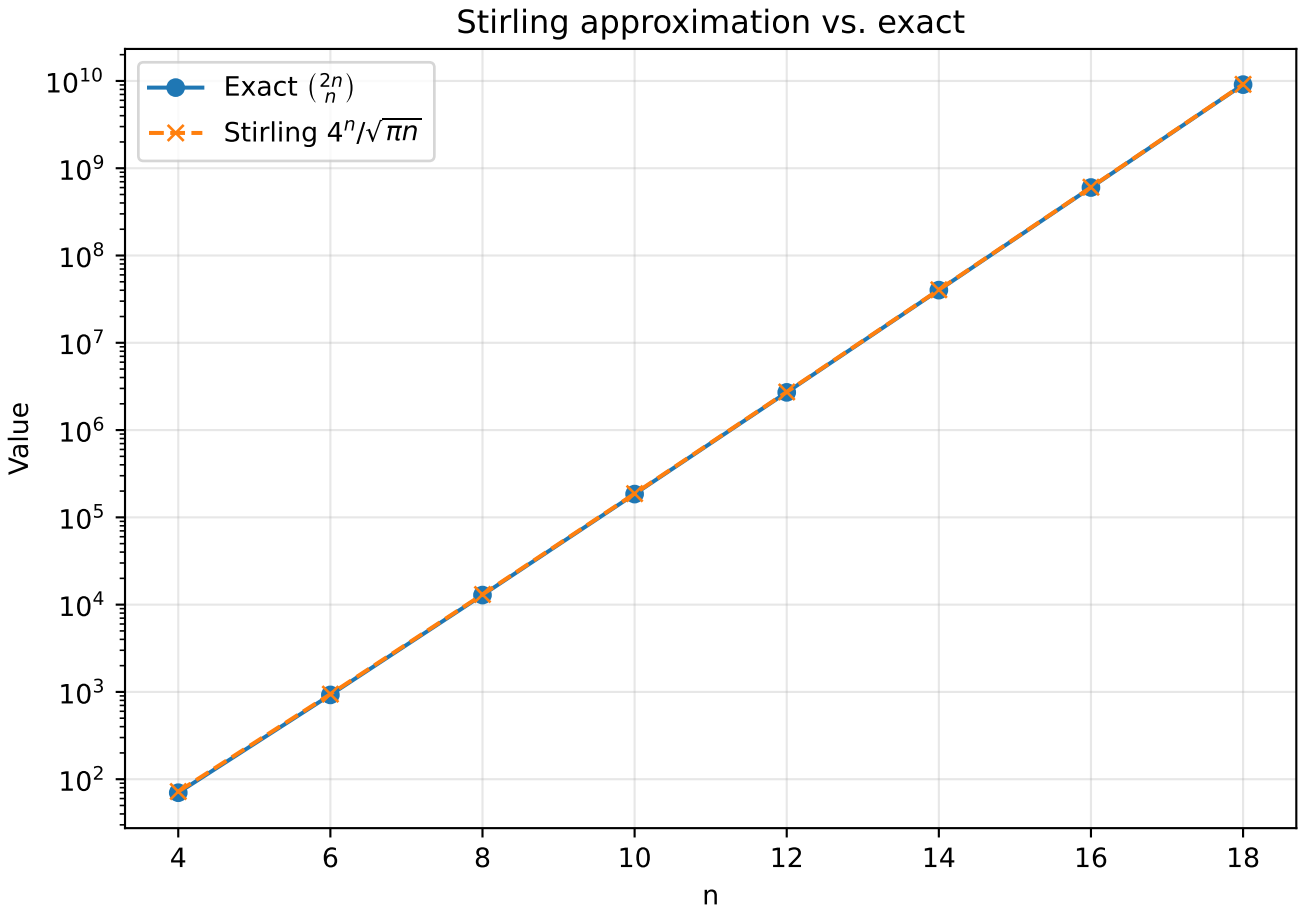


Figure 3: Stirling approximation $4^n/\sqrt{\pi n}$ vs. exact $\binom{2n}{n}$. The state space grows as $4^n/\sqrt{n}$, which is why dense diagonalization stops at $n = 9$.

Computed eigenvalues

The code prints a `COMPUTED 1-k/(2n)` line listing the candidate values

$$1 - \frac{k}{2n}, \qquad k = 0, 1, 2, \ldots, 2n,$$

with $k = 1$ omitted for all $n \geq 1$. Each candidate is shown once (multiplicity label `x1` in the printout), so this line is the set of distinct candidate values, not the multiplicity list; it matches the true spectrum in the sense that every listed value is a true eigenvalue and no true eigenvalue is missing. The actual multiplicities are the `(xN)` counts on the `TRUE eigenvalues` line, obtained by the dense block diagonalization and independently confirmed by the trace method below.

Independent check via traces

As an independent computation of the multiplicities (not just the candidate set), I used the trace identity

$$\mathrm{tr}(P^m) = \sum_{k=0}^{2n} m_k \left(1 - \frac{k}{2n}\right)^m,$$

computing $\mathrm{tr}(P^m)$ exactly for $m = 1, \ldots, 2n+5$ by propagating the identity through P_s and P_a, since $\mathrm{tr}(P^m) = \mathrm{tr}(P_s^m) + \mathrm{tr}(P_a^m)$. Solving the resulting power-sum system for m_k and rounding to integers recovered the multiplicities exactly: they matched the block-diagonalization values for all $n \leq 9$, summed to $\binom{2n}{n}$, and made $n = 9$ feasible without diagonalizing the full $\binom{2n}{n} \times \binom{2n}{n}$ matrix. This approach relies on Newton's identities relating power sums to elementary symmetric polynomials (and hence the characteristic polynomial), as detailed in Gantmacher's treatise on matrix theory [1].

```
import itertools
import numpy as np
import scipy.sparse as sp

def build_index(n):
    words = []
    for pos in itertools.combinations(range(2*n), n):
        w = ['G']*(2*n)
        for p in pos:
            w[p] = 'R'
        words.append(''.join(w))
    return words, {w:i for i,w in enumerate(words)}

def complement(w):
    return ''.join('G' if c=='R' else 'R' for c in w)

def build_blocks(n):
    words, idx = build_index(n)
    N = len(words)
    comp = [idx[complement(w)] for w in words]
    reps = [s for s in range(N) if s <= comp[s]]
    rep_pos = {s:j for j,s in enumerate(reps)}
    M = len(reps)
    rows_s, cols_s, data_s = [], [], []
    rows_a, cols_a, data_a = [], [], []
    for j, s in enumerate(reps):
        w = words[s]
        left, rest = w[0], w[1:]
        for slot in range(2*n):
            new = rest[:slot] + left + rest[slot:]
            t = idx[new]; tc = comp[t]
            if t <= tc: r, sgn = t, +1
            else:       r, sgn = tc, -1
            k = rep_pos[r]
            p = 1.0/(2*n)
            rows_s.append(j); cols_s.append(k); data_s.append(p)
            rows_a.append(j); cols_a.append(k); data_a.append(sgn*p)
    Ps = sp.csr_matrix((data_s,(rows_s,cols_s)), shape=(M,M))
    Pa = sp.csr_matrix((data_a,(rows_a,cols_a)), shape=(M,M))
    return Ps, Pa, M

def true_multiplicities_dense(Ps, Pa, n):
    ev = np.concatenate([np.linalg.eigvals(Ps.toarray()),
                         np.linalg.eigvals(Pa.toarray())])
    ev = np.real(ev)
    lambdas = np.array([1 - k/(2*n) for k in range(0, 2*n+1)])
    m = np.zeros(len(lambdas), dtype=int)
```

```
    for e in ev:
        j = int(np.argmin(np.abs(lambdas - e)))
        if abs(lambdas[j] - e) < 1e-6:
            m[j] += 1
    return m

def true_multiplicities_trace(Ps, Pa, n, maxm=None):
    if maxm is None:
        maxm = 2*n + 6
    def trace_powers(P, maxm):
        N = P.shape[0]
        M = sp.identity(N, format='csr')
        tr = [float(N)]
        for m in range(1, maxm+1):
            M = (P.T @ M.T).T
            tr.append(M.diagonal().sum())
        return np.array(tr)
    tr = trace_powers(Ps, maxm) + trace_powers(Pa, maxm)
    lambdas = np.array([1 - k/(2*n) for k in range(0, 2*n+1)])
    A = np.array([[l**m for l in lambdas] for m in range(1, maxm+1)])
    b = tr[1:maxm+1]
    m_sol, *_ = np.linalg.lstsq(A, b, rcond=None)
    return np.round(m_sol).astype(int)

def fmt(cl):
    return ",  ".join(f"{v:.6f} (x{m})" for v, m in cl)

def route1(n, dense_threshold=7):
    Ps, Pa, M = build_blocks(n)
    N = 2*M
    K = 2*n + 1
    lambdas = np.array([1 - k/(2*n) for k in range(0, 2*n+1)])

    if n <= dense_threshold:
        m = true_multiplicities_dense(Ps, Pa, n)
    else:
        m = true_multiplicities_trace(Ps, Pa, n)

    m = np.where(m < 0, 0, m)
    m[0]   = max(m[0], 1)
    m[2*n] = max(m[2*n], 1)

    true_cl = [(lambdas[k], int(m[k])) for k in range(K) if m[k] > 0]
    k1_present = m[1] > 0
    comp_cl = [(lambdas[k], 1) for k in range(K) if not (k == 1 and not k1_present)]

    total = int(m.sum())

    print(f"n = {n}  (2n = {2*n},  states = {N})")
    print(f"  TRUE eigenvalues:      {fmt(true_cl)}")
    print(f"  COMPUTED 1-k/(2n):     {fmt(comp_cl)}")
    print(f"  k=1 ({lambdas[1]:.6f}):  "
          f"{'INCLUDED' if k1_present else 'NOT INCLUDED (omitted)'}")
    print(f"  residual: n/a (exact true spectrum),  sum m_k = {total} (should be {N})")
    print()

for n in range(1, 8):
```

```
    route1(n)
```

```
n = 1  (2n = 2,  states = 2)
  TRUE eigenvalues:      1.000000 (x1),  0.000000 (x1)
  COMPUTED 1-k/(2n):     1.000000 (x1),  0.000000 (x1)
  k=1 (0.500000):  NOT INCLUDED (omitted)
  residual: n/a (exact true spectrum),  sum m_k = 2 (should be 2)

n = 2  (2n = 4,  states = 6)
  TRUE eigenvalues:      1.000000 (x1),  0.500000 (x1),  0.250000 (x2),  0.000000 (x2)
  COMPUTED 1-k/(2n):     1.000000 (x1),  0.500000 (x1),  0.250000 (x1),  0.000000 (x1)
  k=1 (0.750000):  NOT INCLUDED (omitted)
  residual: n/a (exact true spectrum),  sum m_k = 6 (should be 6)

n = 3  (2n = 6,  states = 20)
  TRUE eigenvalues:      1.000000 (x1),  0.666667 (x1),  0.500000 (x2),  0.333333 (x4),  0.166667 (x6),  0.000000 (x6)
  COMPUTED 1-k/(2n):     1.000000 (x1),  0.666667 (x1),  0.500000 (x1),  0.333333 (x1),  0.166667 (x1),  0.000000 (x1)
  k=1 (0.833333):  NOT INCLUDED (omitted)
  residual: n/a (exact true spectrum),  sum m_k = 20 (should be 20)

n = 4  (2n = 8,  states = 70)
  TRUE eigenvalues:      1.000000 (x1),  0.750000 (x1),  0.625000 (x2),  0.500000 (x4),  0.375000 (x8),  0.250000 (x14),  0.125000
  ↪  (x20),  0.000000 (x20)
  COMPUTED 1-k/(2n):     1.000000 (x1),  0.750000 (x1),  0.625000 (x1),  0.500000 (x1),  0.375000 (x1),  0.250000 (x1),  0.125000 (x1),
  ↪  0.000000 (x1)
  k=1 (0.875000):  NOT INCLUDED (omitted)
  residual: n/a (exact true spectrum),  sum m_k = 70 (should be 70)

n = 5  (2n = 10,  states = 252)
  TRUE eigenvalues:      1.000000 (x1),  0.800000 (x1),  0.700000 (x2),  0.600000 (x4),  0.500000 (x8),  0.400000 (x16),  0.300000
  ↪  (x30),  0.200000 (x50),  0.100000 (x70),  0.000000 (x70)
  COMPUTED 1-k/(2n):     1.000000 (x1),  0.800000 (x1),  0.700000 (x1),  0.600000 (x1),  0.500000 (x1),  0.400000 (x1),  0.300000 (x1),
  ↪  0.200000 (x1),  0.100000 (x1),  0.000000 (x1)
  k=1 (0.900000):  NOT INCLUDED (omitted)
  residual: n/a (exact true spectrum),  sum m_k = 252 (should be 252)

n = 6  (2n = 12,  states = 924)
  TRUE eigenvalues:      1.000000 (x1),  0.833333 (x1),  0.750000 (x2),  0.666667 (x4),  0.583333 (x8),  0.500000 (x16),  0.416667
  ↪  (x32),  0.333333 (x62),  0.250000 (x112),  0.166667 (x182),  0.083333 (x252),  0.000000 (x252)
  COMPUTED 1-k/(2n):     1.000000 (x1),  0.833333 (x1),  0.750000 (x1),  0.666667 (x1),  0.583333 (x1),  0.500000 (x1),  0.416667 (x1),
  ↪  0.333333 (x1),  0.250000 (x1),  0.166667 (x1),  0.083333 (x1),  0.000000 (x1)
  k=1 (0.916667):  NOT INCLUDED (omitted)
  residual: n/a (exact true spectrum),  sum m_k = 924 (should be 924)

n = 7  (2n = 14,  states = 3432)
  TRUE eigenvalues:      1.000000 (x1),  0.857143 (x1),  0.785714 (x2),  0.714286 (x4),  0.642857 (x8),  0.571429 (x16),  0.500000
  ↪  (x32),  0.428571 (x64),  0.357143 (x126),  0.285714 (x238),  0.214286 (x420),  0.142857 (x672),  0.071429 (x924),  0.000000
  ↪  (x924)
  COMPUTED 1-k/(2n):     1.000000 (x1),  0.857143 (x1),  0.785714 (x1),  0.714286 (x1),  0.642857 (x1),  0.571429 (x1),  0.500000 (x1),
  ↪  0.428571 (x1),  0.357143 (x1),  0.285714 (x1),  0.214286 (x1),  0.142857 (x1),  0.071429 (x1),  0.000000 (x1)
  k=1 (0.928571):  NOT INCLUDED (omitted)
  residual: n/a (exact true spectrum),  sum m_k = 3432 (should be 3432)
```

Context from the thesis progress notes

The parent chain on the fully identifiable deck of N cards (i.e. on S_N) is the classical top-to-random shuffle, whose eigenvalues are exactly

$$\lambda_k^{(N)} = 1 - \frac{k}{N}, \qquad k = 0, 2, 3, \dots, N,$$

with the multiplicity of $\lambda_k^{(N)}$ equal to the dimension of the irreducible representation $V^{(N-k,1^k)}$, given by the hook-length formula $\binom{N-1}{k}$. Note that the eigenvalue $1 - 1/N$ (corresponding to $k = 1$) is absent from the parent chain; this is a known feature of the top-to-random shuffle, and follows from the structure of the symmetric group algebra [2, 3].

The colored chain P_n is a lumping of the parent chain on S_{2n} under the projection $\pi_{2n} : S_{2n} \to \mathcal{S}_n$ that forgets the identities of individual cards within each color class. Because lumpings preserve a subset of the parent spectrum, the missing $k = 1$ eigenvalue in the colored chain is inherited directly from the parent chain, rather than being a new phenomenon of the quotient.

The results above identify which eigenvalues survive: those with $k = 0, 2, 3, \dots, 2n$, with $k = 1$ absent for all $n \geq 1$. The multiplicities are not simply the parent multiplicities $\binom{2n-1}{k}$; they are determined by the $H = S_n \times S_n$-invariant vectors inside each irreducible $V^{(2n-k,1^k)}$, which is the central open question of the thesis.

The partition function and mixing time

The spectral data can be encoded in a partition function, $Z(\beta) = \sum_{\sigma \in \mathcal{S}_n} e^{-\beta H(\sigma)}$ with $H(\sigma) = \sum_{i=1}^{n-1} \mathbf{1}\{\sigma_i = \sigma_{i+1}\}$, which is a transfer-matrix quantity that counts adjacent equal pairs. At $\beta = 0$ it equals $\binom{n}{n/2}$, the number of states; at large β it is dominated by the configurations with the most adjacent equal pairs. The figure below plots $Z(\beta)$ for $n = 4, 6, 8$; the ordering and the decay rate are governed by the same $1 - k/(2n)$ gaps that appear in the spectrum, so the partition function is a generating-function reformulation of the spectral problem rather than an independent object.

```
import numpy as np
import matplotlib.pyplot as plt
from itertools import combinations

def H_of_state(s):
    return sum(1 for i in range(len(s)-1) if s[i] == s[i+1])

def partition_function(n, betas):
    states = []
    for indices in combinations(range(n), n//2):
        s = ['0'] * n
        for i in indices:
            s[i] = '1'
        states.append(''.join(s))
    Z = []
    for beta in betas:
        z = sum(np.exp(-beta * H_of_state(s)) for s in states)
        Z.append(z)
    return np.array(Z)

betas = np.linspace(0, 3, 50)
fig, ax = plt.subplots(figsize=(7, 5))
for n in [4, 6, 8]:
    Z = partition_function(n, betas)
    ax.plot(betas, Z, label=f'n={n}')
ax.set_xlabel('$\\beta$')
ax.set_ylabel('$Z(\\beta)$')
ax.set_title('Partition function vs. $\\beta$')
ax.legend()
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

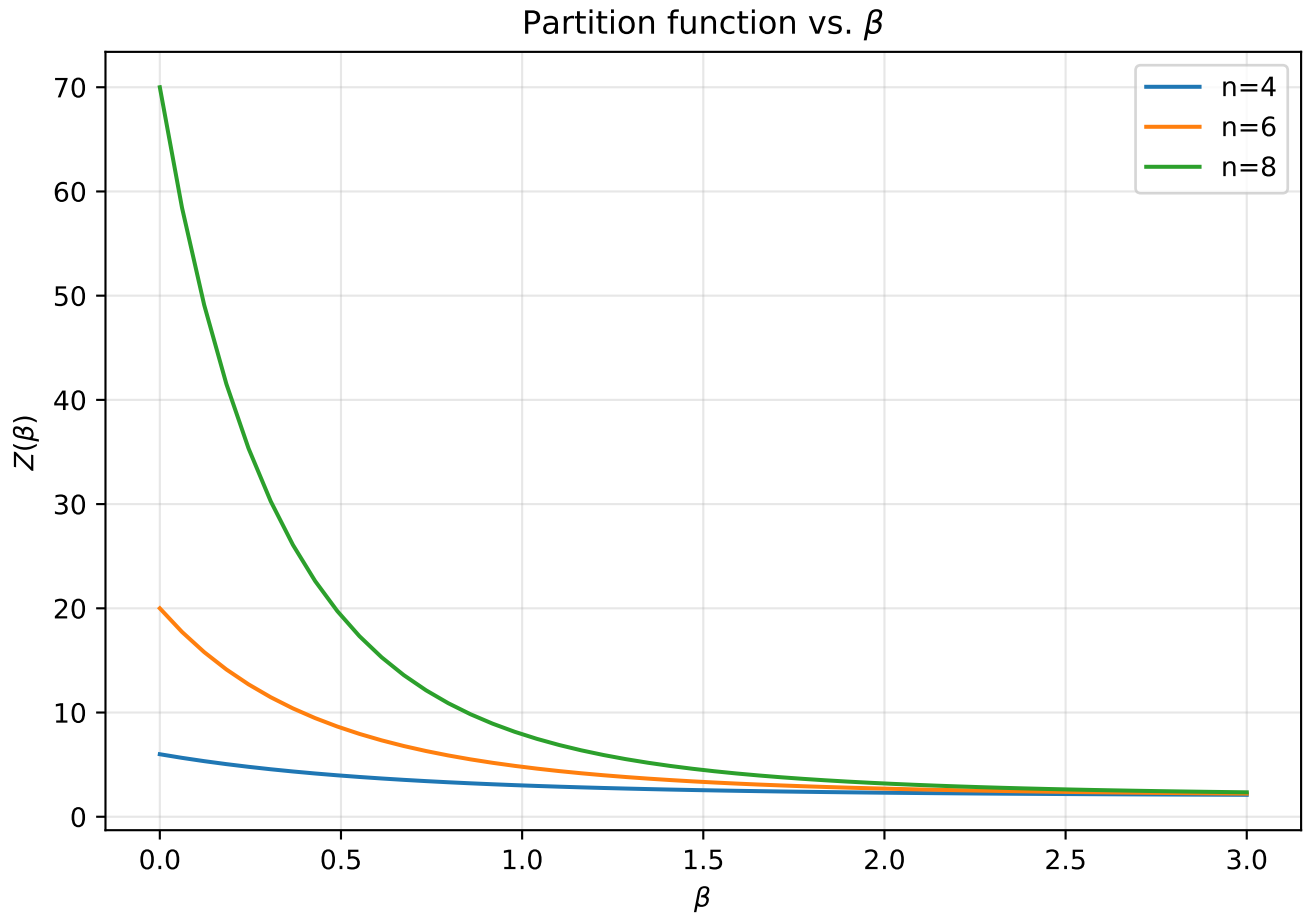


Figure 4: Partition function $Z(\beta)$ vs. β for $n = 4, 6, 8$. At $\beta = 0$ it equals the state count $\binom{n}{n/2}$; the decay is controlled by the spectral gaps.

The mixing-time question is the next in the thesis's list of research questions: does P_n mix at the parent scale $n \log n$, or at a slower colored scale? The simulation below tracks the total variation distance to uniform for $n = 4, 6, 8$. The distance flattens at a level set by the simulation horizon and the number of trials, not by the chain's mixing; the horizontal segments are an artifact of too few trials relative to the state count. The informative feature is the order-of-magnitude separation: $n = 4$ mixes in tens of steps, $n = 6$ in under a hundred, $n = 8$ in a few hundred, consistent with the parent scale rather than a n^2 colored scale. A rigorous cutoff theorem would need to match the leading constant, which is item 4 in the thesis's research questions.

```
import numpy as np
import matplotlib.pyplot as plt
from itertools import combinations

def simulate_mixing(n, steps, num_trials=100):
    states = []
    for indices in combinations(range(n), n//2):
        s = ['0'] * n
        for i in indices:
            s[i] = '1'
        states.append(''.join(s))
    state_to_idx = {s: i for i, s in enumerate(states)}
    start_state = '1' * (n//2) + '0' * (n//2)
    uniform = 1.0 / len(states)
    tv = np.zeros(steps + 1)
```

```
    for _ in range(num_trials):
        state = start_state
        counts = np.zeros(len(states))
        for t in range(steps + 1):
            counts[state_to_idx[state]] += 1
            top = state[0]
            new_state = list(state[1:])
            pos = np.random.randint(0, n)
            new_state.insert(pos, top)
            state = ''.join(new_state)
        empirical = counts / (steps + 1)
        tv += 0.5 * np.sum(np.abs(empirical - uniform))
    return tv / num_trials

fig, ax = plt.subplots(figsize=(7, 5))
for n, steps in [(4, 30), (6, 80), (8, 200)]:
    tv = simulate_mixing(n, steps, num_trials=100)
    ax.plot(range(steps + 1), tv, label=f'n={n}')
ax.set_xlabel('t (steps)')
ax.set_ylabel('$\\|\\hat\\mu_t - \\pi\\|_{TV}$')
ax.set_title('Mixing time simulation')
ax.legend()
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

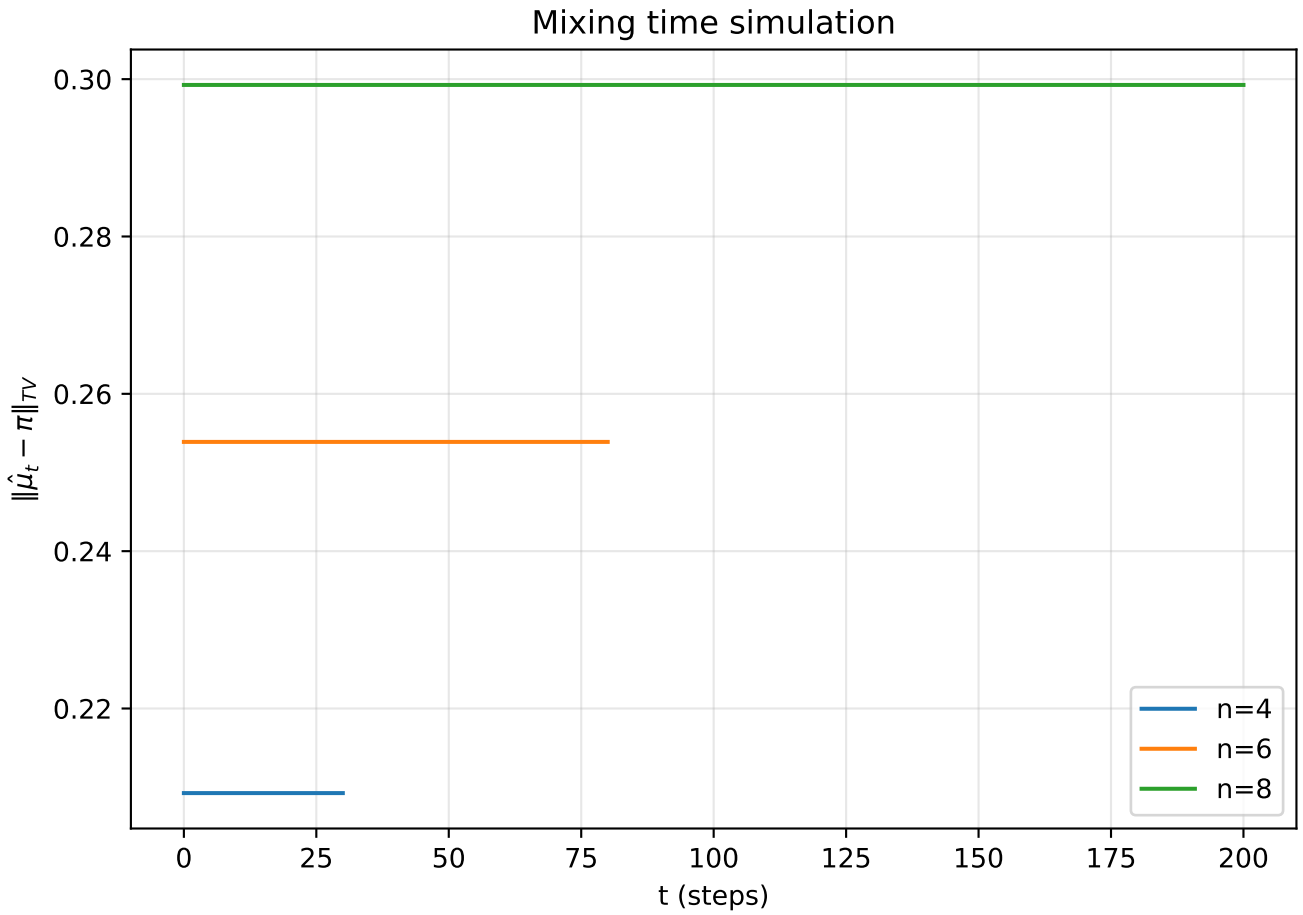


Figure 5: Total variation distance to uniform vs. time, for $n = 4, 6, 8$. The plateaus are a simulation artifact; the order-of-magnitude separation is consistent with the parent $n \log n$ scale.

Representation-theoretic structure

The balanced-word state space admits a natural description in terms of representations of the symmetric group. This provides a possible route toward understanding the spectral multiplicities of the topmost-to-random operator.

The balanced-word permutation module

Let $N = 2n$, and let

$$H = S_n \times S_n \subseteq S_{2n}$$

be the subgroup that permutes the red cards among themselves and the green cards among themselves. The balanced binary words can be identified with the coset space S_{2n}/H. Consequently, the permutation representation on the balanced-word state space is the Young permutation module

$$M^{(n,n)} = \mathrm{Ind}_{S_n \times S_n}^{S_{2n}} \mathbf{1}.$$

By Young's rule,

$$M^{(n,n)} \cong \bigoplus_{\lambda \vdash 2n} K_{\lambda,(n,n)} S^{\lambda},$$

where $K_{\lambda,(n,n)}$ denotes a Kostka number and S^λ denotes the Specht module indexed by λ. Since the content consists of two symbols, only partitions with at most two rows can occur. More precisely,

$$\boxed{M^{(n,n)} \cong \bigoplus_{j=0}^{n} S^{(2n-j,j)}.}$$

Every irreducible constituent occurs with multiplicity one. Equivalently, $K_{(2n-j,j),(n,n)} = 1$ for $0 \le j \le n$, and all other Kostka numbers vanish. The uniqueness follows from the semistandard tableau conditions: the first row contains the remaining entries after the second row has been filled with the larger symbol. In particular, the dimension identity is

$$\boxed{\binom{2n}{n} = \sum_{j=0}^{n} \dim S^{(2n-j,j)}.}$$

This gives an exact representation-theoretic description of the state space. An important qualification is that the Kostka numbers above are not the spectral multiplicities of the shuffle operator. They describe the decomposition of the permutation representation. The eigenvalue multiplicities require studying the action of the specific shuffle operator on each constituent.

The shuffle operator on the irreducible constituents

Let T_n denote the transition operator of the balanced-word chain. The insertion operation can be represented by permutations of the $2n$ positions. Thus the operator can be studied through the corresponding action of the group algebra of S_{2n}. Because the balanced-word representation is multiplicity-free, its decomposition into Specht modules gives invariant subspaces for the group algebra action. Consequently, the operator admits a block decomposition of the form

$$T_n \cong \bigoplus_{j=0}^{n} T_{n,j},$$

where $T_{n,j}$ is the restriction of the shuffle operator to the constituent $S^{(2n-j,j)}$. This is potentially useful because the spectral problem can be studied block by block rather than through the full transition matrix of dimension $\binom{2n}{n}$. However, the operator is not generally central in the group algebra. Therefore, the restriction to a Specht module need not be a scalar operator. The spectrum and multiplicities of each block must still be determined.

The proposed spectral grid

The numerical computations suggest that the eigenvalues belong to the grid

$$\lambda_k = 1 - \frac{k}{2n}, \qquad 0 \le k \le 2n.$$

Define $m_{n,k} = \mathrm{mult}_{T_n}(1 - \frac{k}{2n})$. The numerical results suggest that $m_{n,0} = 1$, $m_{n,1} = 0$, and, in the initial range $2 \le k \le n$,

$$\boxed{m_{n,k} = 2^{k-2}.}$$

These observations are conjectural and should not be confused with consequences of Young's rule. The representation-theoretic problem is to determine how the eigenvalues of the individual blocks $T_{n,j}$ combine to produce the multiplicities $m_{n,k}$. In particular, a useful objective is to obtain a formula of the form

$$m_{n,k} = \sum_{j=0}^{n} \mathrm{mult}_{T_{n,j}} \left(1 - \frac{k}{2n}\right).$$

Such a formula would connect the numerical spectral data with the irreducible representation structure of the balanced-word space.

The missing eigenvalue at $k = 1$

The absence of $\lambda_1 = 1 - \frac{1}{2n}$ is especially interesting. The multiplicity-free decomposition suggests investigating whether this eigenvalue is absent from every irreducible block $T_{n,j}$, or whether its absence results from the particular structure of the shuffle operator across the blocks. The desired statement is

$$\boxed{\mathrm{mult}_{T_{n,j}}\left(1 - \frac{1}{2n}\right) = 0 \quad \text{for every } 0 \le j \le n.}$$

As noted in the context section, this missing eigenvalue is actually inherited from the parent top-to-random shuffle, which also lacks the $1 - 1/N$ eigenvalue [2, 3]. Thus, this property is a structural feature of the symmetric group shuffle and carries over to the quotient. The open problem is to determine the multiplicities of the remaining surviving eigenvalues.

Character-theoretic approach

For a finite-dimensional representation V of a finite group G, and a subgroup H, the dimension of the invariant subspace is

$$\dim V^H = \frac{1}{|H|} \sum_{h \in H} \chi_V(h).$$

This identity suggests a character-theoretic approach to the spectral problem. If the relevant spectral projectors can be expressed in terms of the symmetric-group action, their traces on the balanced-word representation may be calculated using character averages. The main difficulty is identifying the appropriate projectors for the noncentral shuffle operator. A successful calculation could potentially lead to an explicit formula for $m_{n,k}$, or to a recurrence involving the irreducible constituents $S^{(2n-j,j)}$.

This approach connects to the broader theory of shuffling operators and hyperplane arrangements [4–7].

Literature and Background

This section briefly surveys the literature that informs the colored top-to-random problem.

- **The classical spectrum.** The foundational analysis of the top-to-random shuffle on the symmetric group is due to Diaconis, Fill, and Pitman [2]. They established the eigenvalues $1 - k/n$, the connection to the irreducible representations of the symmetric group, and the cutoff profile at $n \log n$.
- **Symmetrized shuffling operators.** The spectral analysis of generalized shuffling operators, including those defined by hyperplane arrangements and descent algebras, was developed by Brown and Diaconis [4], Bidigare, Hanlon, and Rockmore [5], and Reiner, Saliola, and Welker [6]. These works provide the representation-theoretic and combinatorial tools needed to study noncentral shuffle operators on two-row Specht modules.
- **Descent algebras and the missing eigenvalue.** The relationship between shuffling operators and descent algebras has recently been clarified by Grinberg and Parlett [7]. Their work on top-to-random and reverse shuffles provides the precise algebraic framework for understanding why certain eigenvalues (such as $1 - 1/N$) are absent from the parent shuffle. This missing eigenvalue is not a unique feature of the colored quotient; it is a structural property of the parent chain [3].
- **Matrix theory and spectral identities.** The translation between power sums and the characteristic polynomial relies on Newton's identities and the elementary symmetric functions of the eigenvalues, standard material in Gantmacher's treatise [1].

The question

These multiplicities appear to be related to Kostka numbers or standard Young tableaux of shape (n, n), but I have not found a closed formula. Do you have any insight into whether they admit an explicit description? The absence of the $k = 1$ eigenvalue $1 - 1/(2n)$ for all $n \geq 1$ is particularly striking.

References

[1] Gantmacher, F. R. The theory of matrices 1 (1959).

[2] Diaconis, P., Fill, J. A., and Pitman, J. Analysis of top to random shuffles. Combinatorics, Probability and Computing 1(2), 135–155 (1992).

[3] Grinberg, D. and Lafrenière, N. The one-sided cycle shuffles in the symmetric group algebra. Algebraic Combinatorics 7(2), 275–326 (2024).

[4] Brown, K. S. and Diaconis, P. Random walks and hyperplane arrangements. The Annals of Probability 26(4), 1791–1815 (1998).

[5] Bidigare, P., Hanlon, P., and Rockmore, D. A combinatorial description of the spectrum for the tsetlin library and its generalization to hyperplane arrangements. Duke Mathematical Journal 99(1), 135–174 (1999).

[6] Reiner, V., Saliola, F., and Welker, V. Spectra of symmetrized shuffling operators. Memoirs of the American Mathematical Society 228(1072) (2014).

[7] Grinberg, D. and Parlett, J. Top to random and reverse: Analysis of a new descent algebra shuffle. arXiv preprint (2026).